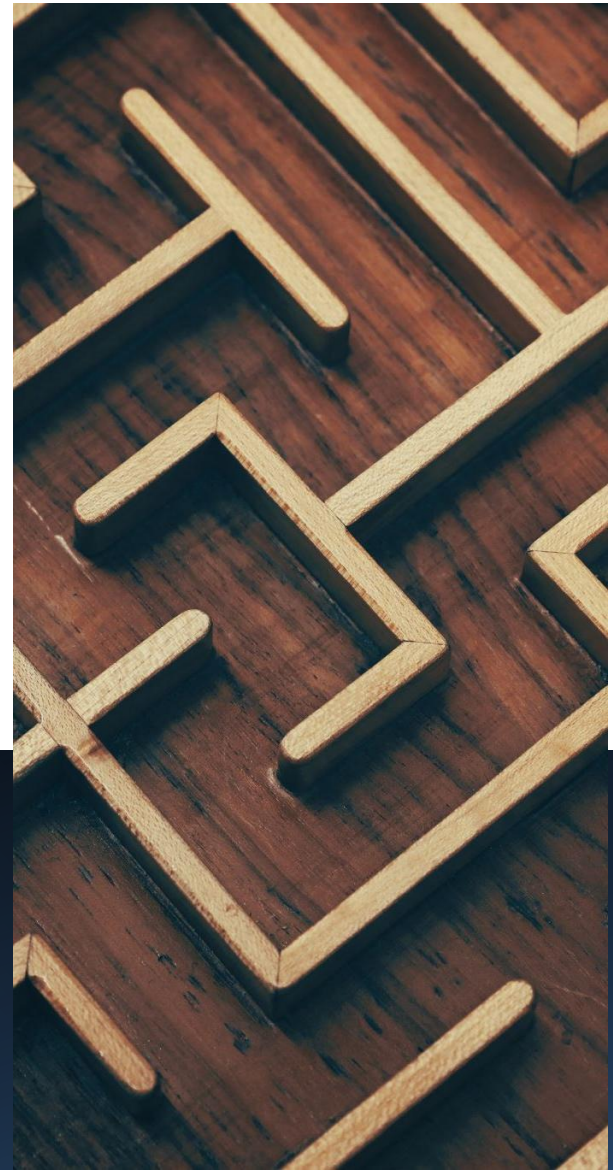


Maze Solver Using AI Search Algorithms BFS, DFS & A*

Yalda Rawan
Artificial Intelligence



Problem

- Find a path from Start (S) to Goal (G)
- Grid-based maze
- Obstacles (#) block movement

```
[ 'S', '.', '.', '.', '#', '.', '.', '.']  
[ '#', '#', '#', '.', '#', '.', '#', '.']  
[ '.', '.', '.', '.', '.', '.', '#', '.']  
[ '.', '#', '#', '#', '#', '.', '#', '.']  
[ '.', '.', '.', '.', '#', '.', '.', '.']  
[ '#', '#', '#', '.', '#', '#', '#', '.']  
[ '.', '.', '.', '.', '.', '.', '.', '.']  
[ '.', '#', '#', '#', '#', '#', '#', 'G']
```

Algorithms Used

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- A* Search



Maze Setup

Maze Representation:

- S = Start
- G = Goal
- # = Wall
- . = Open path

```
# Step 2: Create a Larger maze
```

```
# S = Start
```

```
# G = Goal
```

```
# . = Open path
```

```
# # = Wall / obstacle
```

```
maze = [  
    ['S', '.', '.', '.', '#', '.', '.', '.'],  
    ['#', '#', '#', '.', '#', '.', '#', '.'],  
    ['.', '.', '.', '.', '.', '.', '#', '.'],  
    ['.', '#', '#', '#', '#', '.', '#', '.'],  
    ['.', '.', '.', '.', '#', '.', '.', '.'],  
    ['#', '#', '#', '.', '#', '#', '#', '.'],  
    ['.', '.', '.', '.', '.', '.', '.', '.'],  
    ['.', '#', '#', '#', '#', '#', '#', 'G']  
]
```

```
rows = len(maze)
```

```
cols = len(maze[0])
```

```
for row in maze:  
    print(row)
```

BFS

Breadth-First Search:

- Uses a queue
- Explores level by level
- Finds the shortest path

```
# Step 6: Breadth-First Search (BFS)

def bfs(start, goal):
    # BFS explores the maze Level by Level
    # It uses a queue and usually finds the shortest path

    start_time = time.perf_counter()

    queue = deque()
    queue.append((start, [start]))

    visited = set()
    explored_nodes = 0

    while queue:
        current, path = queue.popleft()

        if current in visited:
            continue

        visited.add(current)
        explored_nodes += 1

        if current == goal:
            end_time = time.perf_counter()
            return {
                "algorithm": "BFS",
                "path": path,
                "path_length": len(path),
                "explored_nodes": explored_nodes,
                "runtime": end_time - start_time
            }
    }
```

DFS

Depth-First Search:

- Uses a stack
- Explores deeply first
- Not always the shortest path

```
# Step 7: Depth-First Search (DFS)

def dfs(start, goal):
    # DFS explores one path deeply first before backtracking

    start_time = time.perf_counter()

    stack = []
    stack.append((start, [start]))

    visited = set()
    explored_nodes = 0

    while stack:
        current, path = stack.pop()

        if current in visited:
            continue

        visited.add(current)
        explored_nodes += 1

        if current == goal:
            end_time = time.perf_counter()
            return {
                "algorithm": "DFS",
                "path": path,
                "path_length": len(path),
                "explored_nodes": explored_nodes,
                "runtime": end_time - start_time
            }

    # Reversed order helps show that DFS depends on exploration order
```

A*

Algorithm

A* Search:

- Uses heuristic (Manhattan distance)
- Combines cost + estimate
- More efficient

```
# Step 8: Manhattan distance heuristic for A*
```

```
def manhattan_distance(position, goal):
```

```
# Manhattan distance estimates how far a position is from the goal
```

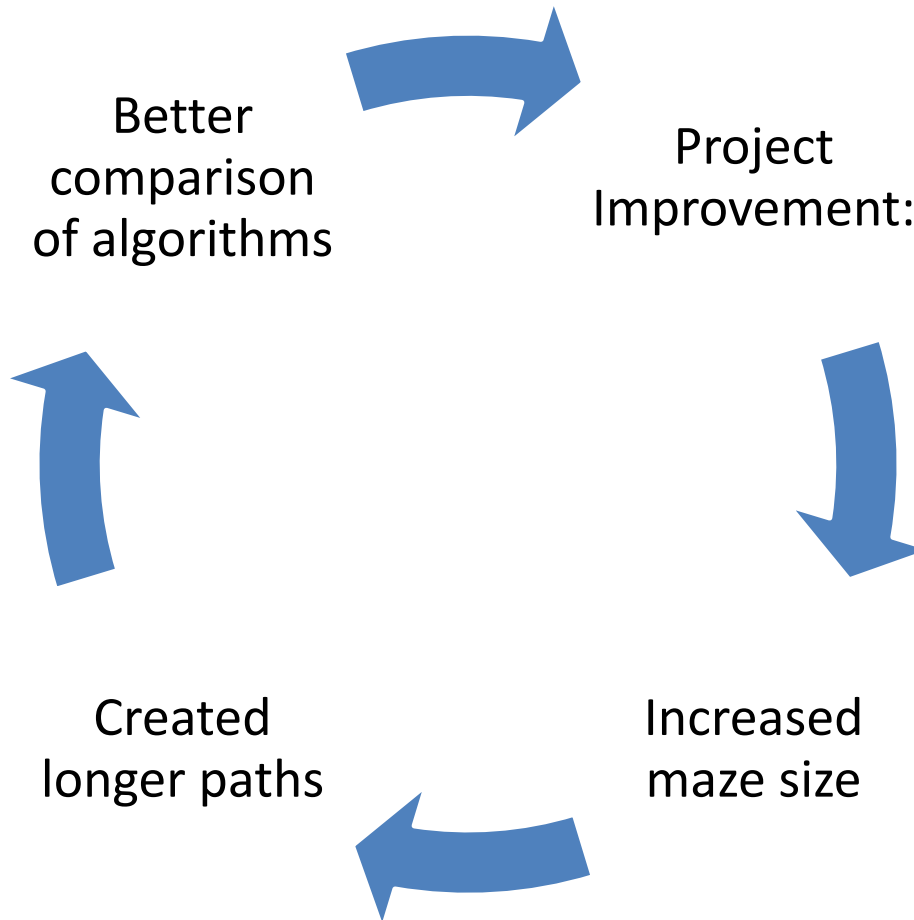
```
# It works well for grid mazes where movement is up, down, left, and right
```

```
x1, y1 = position
```

```
x2, y2 = goal
```

```
return abs(x1 - x2) + abs(y1 - y2)
```

Improvements After 1st Check-in



Results & Analysis

Results Comparison:

- Path Length
- Nodes Explored
- Runtime

BFS

Path: [(0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (4, 5), (4, 6), (4, 7), (5, 7), (6, 7), (7, 7)]

Path length: 15

Explored nodes: 31

Runtime: 8.213e-05 seconds

DFS

Path: [(0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (2, 3), (2, 2), (2, 1), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (5, 3), (6, 3), (6, 4), (6, 5), (6, 6), (6, 7), (7, 7)]

Path length: 21

Explored nodes: 39

Runtime: 9.998e-05 seconds

A*

Path: [(0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (4, 5), (4, 6), (4, 7), (5, 7), (6, 7), (7, 7)]

Path length: 15

Explored nodes: 15

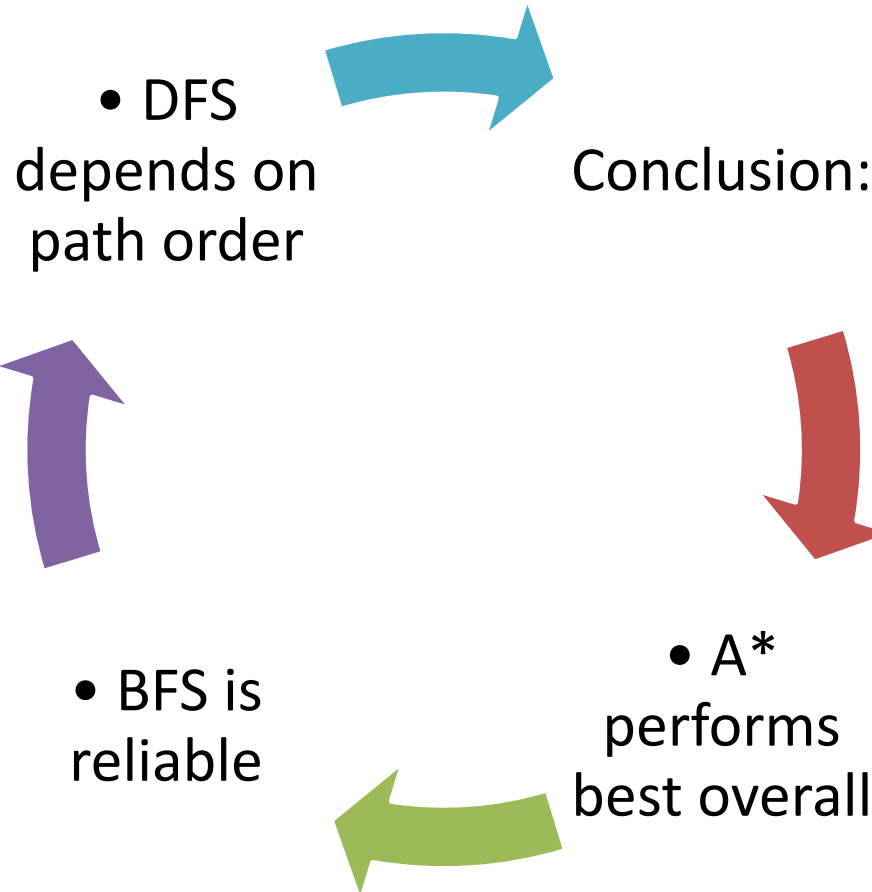
Runtime: 5.516e-05 seconds

Conclusion

+

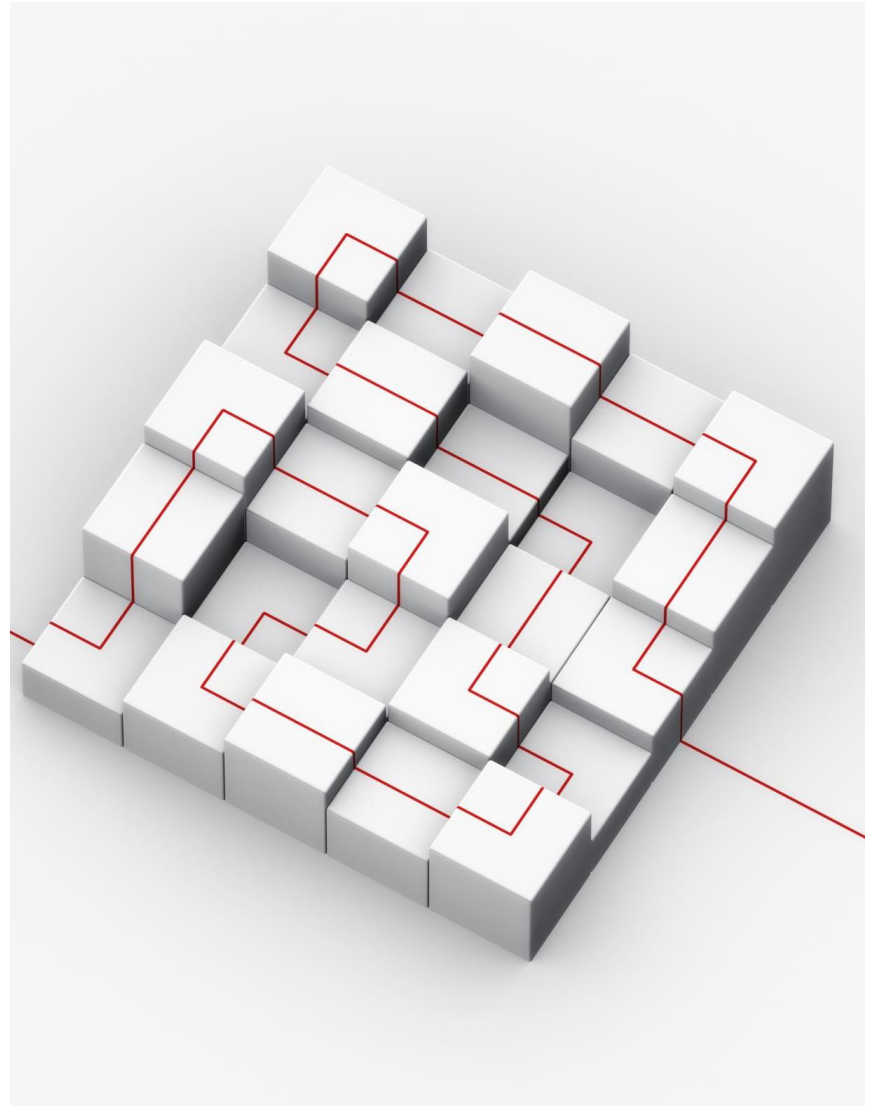
•

0



Future Work

- Test on larger and more complex mazes
- Add more obstacles for realistic scenarios
- Compare different heuristics (Manhattan vs Euclidean)
- Add visualization/animation of the search process
- Apply to real-world problems (GPS, robotics)



Thank you😊